

How Hardware Gets Hacked (Part 5)

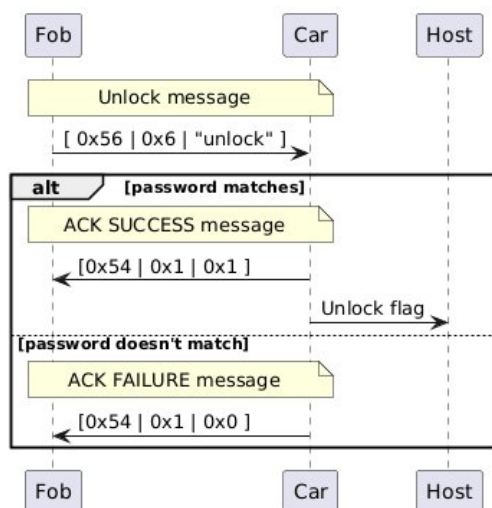
Replay Attacks

by Nathan Jones

Introduction.....	1
Attack #2: Replay attack.....	2
The fundamental problem.....	6
Defense #2a: LCG/PRNG (a near miss).....	9
Conclusion.....	12

Introduction

Although an unlocked debug port was clearly the largest threat to our system in [the last article](#), a cursory glance at the unlocking process in the source code may have had some of you screaming, “The unlock message is literally [0x56 | 0x6 | ‘u’ | ‘n’ | ‘1’ | ‘o’ | ‘c’ | ‘k’]?! “*Any device can successfully send any unlock message to any car!*”



Indeed, they can! (Don’t act *too* incredulous, though; this was almost exactly how key fobs and garage door openers worked until the mid-1980s!)

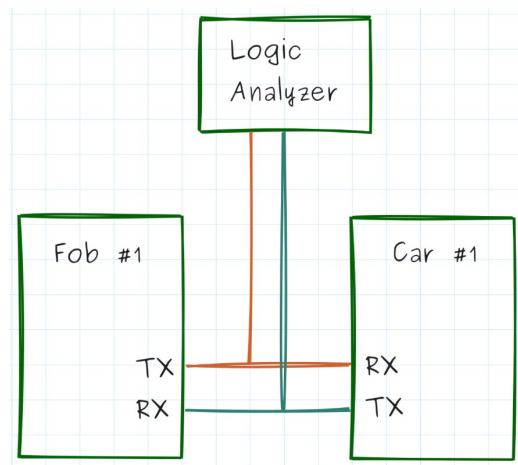
That being said, this is still an egregious vulnerability and its exactly the one to which we’ll direct our attention in this article. Analyzing the specifics of how to attack that and how to

implement a minimum viable defense will introduce us to the topics of replay attacks, authenticity, and the beginnings of cryptography. Let's get started!

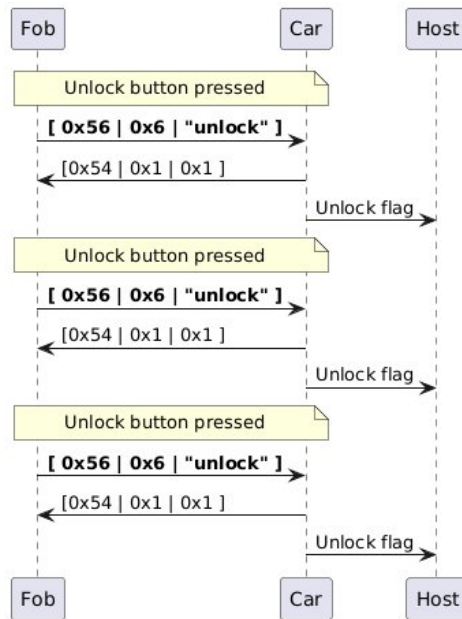
Attack #2: Replay attack

We'll begin by clarifying and conducting the actual attack. The first step requires the attacker to determine that, indeed, a car (any of them) is unlocked by receiving the message [0x56 | 0x6 | 'u' | 'n' | 'l' | 'o' | 'c' | 'k'] on its board UART port. An attacker could have identified that in one of two ways.

Perhaps most straightforwardly, an attacker could have observed a series of unlocking actions by attaching a logic analyzer to the UART pins.



Seeing the same unlock message sent over and over again would have been sufficient to determine that it is only this exact UART message that is needed to unlock one of our cars.



Alternatively, an attacker could have conducted a close analysis of the source code and worked out the nature of the unlock message. For example, lines 436-440 of `fob.c` clearly show how that an unlock message is formed by the `UNLOCK_MAGIC` value (`0x56`), the length of `pair_info.password`, and the value of `pair_info.password`.

```
// fob.c
MESSAGE_PACKET message;
message.message_len = strlen(fob_state_ram->pair_info.password) + 1;
message.magic = UNLOCK_MAGIC;
message.buffer = (uint8_t *)&fob_state_ram->pair_info.password;
send_board_message(&message);

// messages.h
#define UNLOCK_MAGIC 0x56
```

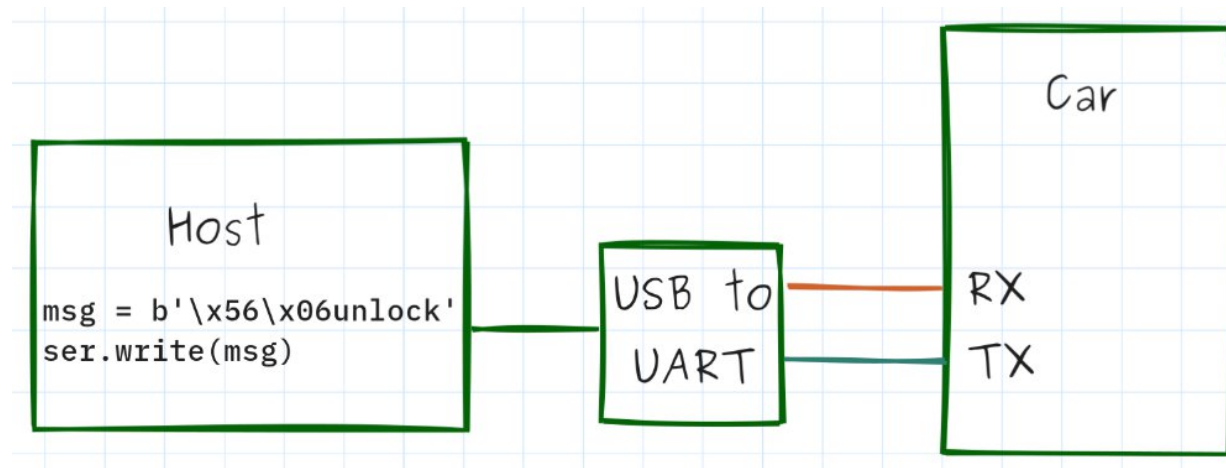
`fob_info.password` gets its value from `PASSWORD` (earlier in `fob.c`), which is defined in `fob_gen_secret.py` as “unlock”.

```
// fob.c
strcpy(fob_state_ram.pair_info.password, PASSWORD);

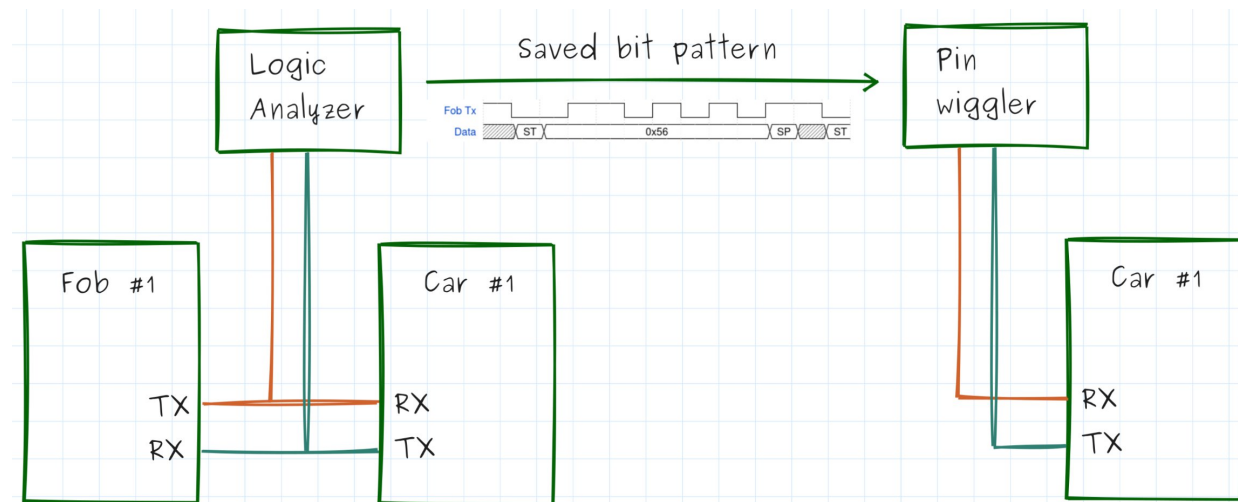
// fob_gen_secret.py
fp.write('#define PASSWORD "unlock"\n\n')
```

Armed with this knowledge, the attack could have conducted the attack in any of three ways.

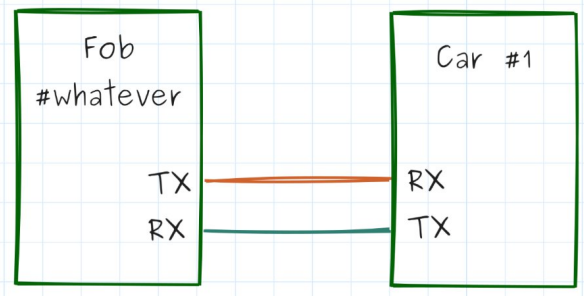
First, the attacker could simply construct a valid unlock message on their computer and send it to a car over a USB-to-UART adapter.



Second, the attacker could make a literal recording of a fob sending out an unlock message and then replay that recording later on. This is known as a **replay attack**.



Third, an attacker could simply attach any fob (such as the fob that comes with Car #0) to the target car and trigger an unlock action, since they all work the same.



I was able to automate the last two attacks with the help of two new test commands: [sendBoardMsg](#) and [getBoardMsgLog](#). The second one returns the last fifteen messages to be sent or received by a board (allowing us to peek at the most recent unlock transaction) and the first one forces a board to send out a given message over it's board UART port (allowing us to force a fob to replay an unlock message that it's just sent).

The two new tests in [testing/security_tests.py](#) are:

- `test_replay_captured_unlock_fails`: Creates a paired fob and car and triggers the fob's button press. Uses `getBoardMsgLog` to retrieve the last unlock and start messages and uses `sendBoardMsg` to replay those same messages.
- `test_fob_paired_to_different_car_cannot_unlock`: Creates a mismatched car and paired fob pair and triggers the fob's button press.

When I run the new security tests, our current firmware fails!

```

(hhghVenv) nathancharlesjones@GeorgeWashingtonCarver:~/Documents/Work/DigiKey/2026/articles/howHardwareGetsHacked_Part2/code$ \pytest testing/security_tests.py -v
===== test session starts =====
platform linux -- Python 3.11.15, pytest-9.0.2, pluggy-1.6.0 -- /home/nathancharlesjones/Documents/Work/DigiKey/2026/articles/howHardwareGetsHacked_Part2/code/hhghVenv/bin/python3.11
cachedir: .pytest_cache
rootdir: /home/nathancharlesjones/Documents/Work/DigiKey/2026/articles/howHardwareGetsHacked_Part2/code
collected 2 items

testing/security_tests.py::TestReplayAttacks::test_replay_captured_unlock_succeeds FAILED [ 50%]
testing/security_tests.py::TestReplayAttacks::test_fob_paired_to_different_car_can_still_unlock FAILED [100%]
  
```

Replaying the unlock messages like this allows us to capture the unlock flag for **every** car, from **Car #1-4**.

—[✓ EXERCISE]—

Which of the following best describes a replay attack? (Answerⁱ)

- An attacker modifies a message in transit before it reaches the receiver
- An attacker decrypts a message by guessing the encryption key
- An attacker records a valid message and retransmits it later

- d) An attacker impersonates a trusted party by stealing their identity
-

The fundamental problem

Before we dive into some defenses, let's take a step back and think about this attack critically. The heart of our current problem lies in the fact that the car, upon receiving an unlock message, has *no confidence* that it wasn't generated by an attacker or another fob. Think about it from the car's perspective: an unlock message can show up at any time, but how does it know who's on the other side? Our system needs a way for the car to be able to say, "Yup, this message *definitely* came from one of my paired fobs."



The security term for "I *do* know who this message came from" is **authenticity**. A message is *authenticated* if a receiver can verify who sent it.

—[? REFLECTION]—

How do you authenticate things in your life?

For example, how would you authenticate an unknown caller on the phone? A person you bumped into at the grocery store who claimed to be a long-lost half-sibling?

Think about your answer before moving on.

It's interesting to me that this stands separate from two other important security qualities: **confidentiality** and **integrity**. A confidential message is one that can't be read by attackers, which can be achieved with encryption. For a message to have integrity, a

receiver needs to have a way to know that the message wasn't tampered with by an attacker en route to the receiver.

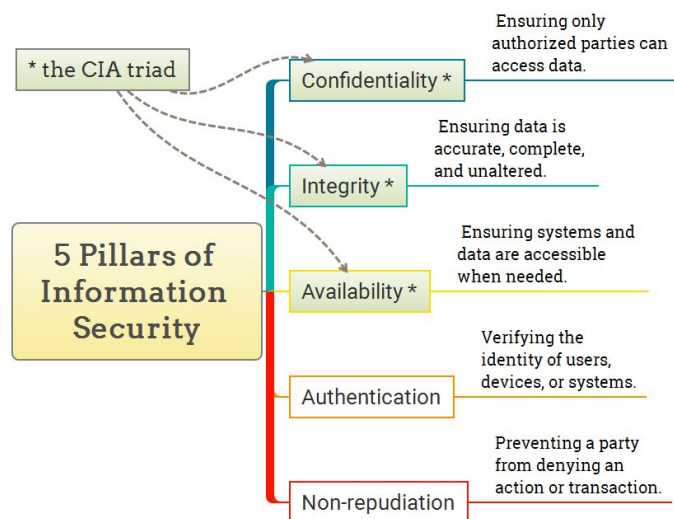


Figure 1: https://miro.medium.com/v2/1*tUBjDPi_f0Zlg7rQb8FI6w.png

In our case, we don't actually much care if an attacker can see the unlock message (i.e. we don't need confidentiality) or even if they modify it before it gets to the car (i.e. we don't need integrity). Neither really presents a concern so long as the car can still authenticate an unlock message from a paired fob when it receives one.



The key difference here is that our communication channel is *adversarial*; an attacker can see every message that goes "across the wire" from fob to car and back again. Like we saw in the actual attack steps, once an attacker knows the special password needed to unlock a

car then the password ceases to be a valid form of authentication. The fact that it works for your bank account is only due to the fact that you've been connecting to the bank's website over HTTPS, which is a secure connection.

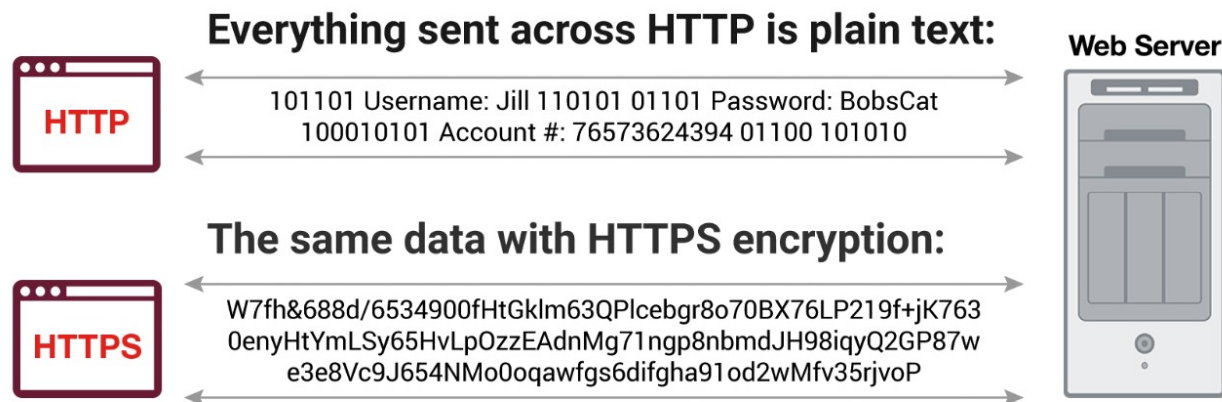


Figure 2: <https://www.fortinet.com/content/dam/fortinet/images/cyberglossary/http-vs-https.jpg>

Before their website ever loaded in your web browser, the bank and your browser were doing a complex dance to be able to send encrypted communications back and forth, the result of which is to prevent an attacker from being able to easily listen in as you send over your login credentials. Without that protection, you'd be vulnerable to things like the infamous "coffee shop attack", in which an attacker in the same coffee shop as you can see your unencrypted internet traffic when you connect to your bank over HTTP as opposed to HTTPS, easily seeing your login credentials.

—[✓ EXERCISE]—

Fill in the blanks using the word bank below. (Answersⁱⁱ)

- A message is **authenticated** if a receiver can verify who ____ it.
- A **confidential** message is one that can't be ____ by attackers, which can be achieved with ____.
- For a message to have **integrity**, a receiver needs to have a way to know that the message wasn't ____ by an attacker ____ to the receiver.

Word bank: *read, sent, encryption, en route, hashed, modified, decrypted, in transit, delivered*

So we need a solution that **allows a car to authenticate a message** as being from a paired fob (or not). Further, this solution needs to work **even if an attacker can see each**

previous message after it's transmitted (i.e. its resistant to replay attacks) *and* our **source code**.

Defense #2a: LCG/PRNG (a near miss)

Let's start by ruling out some "obvious" solutions that won't fix the fundamental problem.

- Encrypt the password: instead of sending
[0x56 | 0x6 | 'u' | 'n' | 'l' | 'o' | 'c' | 'k']
to unlock the car, send
[0x56 | 0x6 | 0xA1 | 0x5B | 0x32 | 0x39 | 0x4D | 0xE5]
instead (or whatever "unlock" would become once it was encrypted).
- Give each fob a unique ID and send that alongside the unlock message, e.g.
[0x56 | 0x8 | 0x0C | 0xED | 'u' | 'n' | 'l' | 'o' | 'c' | 'k']
(in this case, the fob's ID is 0x0CED), so that the car knows who sent the message.

Both of these fixes, unfortunately, still fall victim to the attacks above. In both cases an attacker can still simply replay a valid unlock message to get a car to unlock. (It doesn't matter if the password is encrypted, the attacker can simply replay the encrypted bytes to the same effect.)

In reality what we need is a **"secret" that only the fob and car know**, which the fob can **send to the car** to prove it's who it says it is. Furthermore, this "secret" **can't be revealed during the communication** (so an attacker who's listening to the conversation can't determine it's value). And, of course, the **messages sent have to change** on every subsequent unlock request (to thwart replay attacks).

Can you think of any solutions?

I don't fault you if you can't; we're dipping our toes into the realm of cryptography, here, and its waters are murky.

Here's one solution that we might come up with: use a linear congruential generator (LCG) or pseudo-random number generator (PRNG; aka `rand()`) to generate pseudo-random numbers to send in place of the "unlock" password.

```

// fob.c
MESSAGE_PACKET message;
message.message_len = sizeof(uint32_t);
message.magic = UNLOCK_MAGIC;
uint32_t lcg_value = rand();
message.buffer = (uint8_t *)&lcg_value;
send_board_message(&message);

// LCG parameters
#define A 1664525UL
#define C 1013904223UL

static uint32_t state = 0;

static inline void seed(uint32_t val) { state = val; }

static inline uint32_t next(uint32_t current)
{
    return (uint32_t)(A * current + C);
}

static inline uint32_t rand(void)
{
    state = next(state);
    return state;
}

```

The message format might look something like the one below, where 0x0CED is the fob ID (for multiple fobs to each unlock a car, the car will need to keep track of the current sequence number for each fob by its unique identifier) and 0xA15B3239 is representative of the value produced by `rand()`, our LCG.

[0x56 | 0x6 | 0x0C | 0xED | 0xA1 | 0x5B | 0x32 | 0x39]

During build-time, three values are generated randomly: the seed and the constants A and C. Randomly generating these values at build-time prevents an attacker from determining the value by looking at our source code, as they could do by reading “unlock” in `fob_gen_secret.py`, above. Both the fob and the car then use those values to generate pseudo-random numbers.

Key to this scheme working is that the fob and car are generating the *same* series of random numbers. If the car receives the next value in the sequence from the fob (or, in reality, one of the numbers in a small window of the next possible numbers, which accounts for the fact that a normal person might accidentally press the unlock button while being out of range for the car to hear), then it unlocks.

```

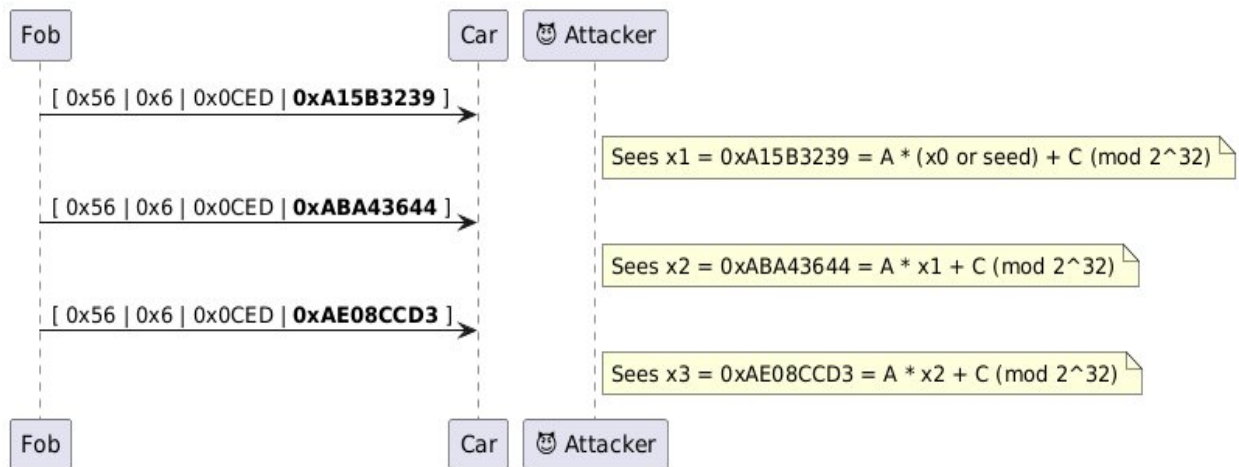
tmp = current car sequence number
FROM 0 to MAX_WINDOW:
    IF tmp == received value:
        UNLOCK
    ELSE:
        tmp = next(tmp)

```

Does this approach satisfy our requirements?

- The attacker never sees or hears the “secret” (i.e. the values of the seed and constants A and C injected at build-time), even if they were watching every unlock message sent by a fob to a car
- The car rejects old codes, preventing simple replay attacks
- It seems that the only way a person or device could determine the next pseudo-random value in the sequence is to know all three of the random values that were generated at build-time, which are only known to the fob and car.

This seems so close! In fact, key fobs and garage door openers moved off of static codes to essentially this scheme in the mid- to late-1980s. But, alas, there is a glaring vulnerability that allows an attacker to ultimately predict every future value of the pseudo-random sequence. It starts with an attacker listening to three consecutive unlock messages.



There's a clear algebraic relationship between the three values the attacker overheard, with three equations and three unknowns. Given the nature of the PRNG in our source code, all it takes is a little algebra to go from the three consecutive values to the values of the constants A and C, allowing an attacker to correctly predict every future message the fob will ever produce.

fancy math, a little algebra, carry the 2 aaaand.....

$$A = (x3 - x2) * \text{modinv}(x2 - x1, 2^{32}) \text{ mod}(2^{32})$$

$$C = x2 - A * x1 \text{ mod}(2^{32})$$

—[✓ EXERCISE]—

What was the sequence number that came immediately before x1 in the picture above?

In other words, what was the seed value for that sequence of numbers (assuming x1 was the first number that was generated)? (Don't feel like you need to solve this by hand; I asked Claude to write me a Python scrip to do it!)

The LCG/PRNG fails because its a *pseudo*-random number generator, with an emphasis on “pseudo”: after observing only three consecutive outputs, anybody with high school math experience can see the deterministic nature of all subsequent values. What we need is a **pseudo-randomness** that approaches **true randomness**, a mathematical operation whose outputs look truly random to anybody who doesn't know the “secret”; in other words, it must be a “one way” operation, one for which the inputs cannot be determined by simply doing the reverse operation on the outputs. In short, we need a **cryptographically secure** mathematical operation, which we'll discuss in the next article!

Conclusion

—[? REFLECTION]—

What's one thing you want to remember about this article?

Write it down or say it to yourself in your head before you move on.

A car that receives a generic unlock message ([0x56 | 0x6 | “unlock”]) has no guarantee that this message came from a correctly paired fob; in other words, the message lacks authenticity. Attackers can easily see and replicate a valid unlock message by just observing the communication channel (or reading the source code!). This is broadly known as a “replay attack”.

Achieving authenticity requires several critical factors:

- That the car and any paired fobs share a “secret” only they know
- That the secret cannot be something that is revealed during unlocking, given the adversarial nature of our communication channel
- That messages must change every time a fob tries to unlock

One solution is to store any PRNG constants and seed values in both the fob and car at build-time and then send the next pseudo-random number in the sequence with each unlock message. E.g.

UNLOCK_MAGIC

	LEN	Fob ID	Pseudo-random #
V	V	/——\	/————\
[0x56	0x6	0x0CED	0xA15B3239]

Although this gives each device the ability to generate a sequence of pseudo-random numbers that only they seem to know, an attacker who observes three consecutive messages (and has some high school math) can easily deduce the “secrets”! What’s needed is a one-way operation, something that won’t let an attacker determine the inputs (i.e. the secrets) just by observing the outputs. In other words, we need a cryptographically secure math operation (which will be revealed in the next article!).

If you’ve made it this far, thanks for reading and happy hacking!

ⁱ Answer: C

ⁱⁱ A message is authenticated if a receiver can verify who sent it. / A confidential message is one that can't be read by attackers, which can be achieved with encryption. / For a message to have integrity, a receiver needs to have a way to know that the message wasn't modified by an attacker in transit to the receiver.